

ISYS2092|3397 | EEET2578 Assignment 03

Selenium / JMeter / BDD / Test Management

1. OVERVIEW

This is a group assessment for teams of 05 students. Your team acts as an independent test consultancy engaged to assess the next release of the Pet Clinic, the same application you tested in Assignment 01 and inspected in Assignment 02. You plan the engagement, coordinate the work in Jira, test the release candidate with Selenium, Katalon BDD, and JMeter, and deliver a release recommendation to the client. You then present that recommendation to the client in person.

The work is deliberately larger than one person can do well. Your team must divide it, track it, and bring the pieces back together into a single coherent assessment. Marks come from the quality of your judgement and from how well the pieces fit, not from the number of artefacts you produce. A submission made of four unrelated pieces stapled together will score poorly even if each piece is competent.

You may use AI tools throughout this assignment. AI use is permitted and encouraged. You must declare what you used and critique it, as described in Section 4.8. Every member must be able to explain and justify any part of the submission, and the client questions in Section 4.7 are where that is checked.

2. PRODUCT REQUIREMENTS

2.1. Client Brief

The Pet Clinic operates a web application used daily by its front desk staff to manage owners, pets, visits, and the services billed on each visit. The clinic has prepared a Release 2.0 candidate. Some of the issues raised in the previous round of testing have been addressed and others may remain. The clinic has not recorded which is which. The clinic wants an independent assessment before the release goes live. In particular it wants confidence in the following:

- The core workflows its staff use every day still behave as specified, especially Visit Services and Billing, where an error has a direct financial effect on customers.
- The features described in its requirements are acceptable to the business, expressed in language the clinic manager can read and confirm.
- The application remains usable when several staff members work in it at the same time.

The clinic expects a clear recommendation on whether to release, supported by evidence, and it expects to hear that recommendation presented by the team that produced it.

2.2. Incomplete Brief

The brief above is not intended to be a complete requirements document, as real client briefs rarely are. It outlines what the clinic cares about, but your team is responsible for defining the specific scope of the engagement.

Where the brief is silent, a test lead should make a reasonable assumption, record it, and proceed. Section 4.1 requires you to document each assumption, explain your reasoning, and state what would change if it proved incorrect. The quality of your assumptions will be assessed. Do not leave your scope undefined simply because the brief does not address a particular point.

3. SYSTEM UNDER TEST

3.1. Application

The system under test is the Pet Clinic Release 2.0 candidate. It covers the same functional areas as Assignment 01: Owner Management, Pet Management, Visit Management, Visit Services and Billing, Veterinarian Directory, and Search With Pagination.

The Requirements Summary issued for Assignment 01 remains the specification of record. Where the release candidate behaves differently from that document, you have identified a candidate defect. The client has not specified which previously reported issues were fixed in this release, so part of your job is to determine what has been fixed and verify that those fixes have not introduced new problems.

3.2. How to Run

The release candidate will be published on Canvas as a runnable application file later. **Sections 4.1 and 4.5 can and should be started BEFORE the release candidate becomes available.**

- You need Java 17 or later installed.
- Run the application with: `java -jar <filename>.jar`
- By default it runs on port 8080, so open `http://localhost:8080` in your browser.
- You can use any free port instead by adding `--server.port=<port>`. This helps if another program is already using 8080.
- The application uses an in-memory database that is reset each time you restart it, which is useful for a clean test run and important for repeatable automation.

This is particularly relevant to Section 4.4. The application runs as a single local process on your own machine, using an in-memory database. Any load figures you produce therefore reflect that environment rather than a production deployment, which is why Section 4.4 requires you to describe your test environment.

4. TASKS

Follow the report structure in Section 05. Each task below maps to a report section and the rubric criterion of the same name. Markers grade each section separately, so make sure your work appears in the appropriate section.

4.1. Test Planning and Assumptions

Produce a test plan for the engagement, covering the following:

- **Scope:** What is in scope and what is explicitly out of scope, with a reason for each exclusion.
- **Risk and Priority:** The product risks you consider most serious for this clinic, and the order in which you will test, justified by those risks.
- **Approach:** The test levels and test types you will apply, and which tool covers which part.
- **Entry and Exit Criteria:** Stated clearly enough to determine whether they have been met.
- **Roles and Schedule:** Who is responsible for each area, and when each part happens.

Include an assumption table with at least 04 entries. Each entry states the assumption, why you made it, and what would change if the assumption turned out to be wrong.

Your exit criteria matter beyond this section. Section 4.6 requires you to assess the release against the exit criteria you define here, so vague criteria such as “all major bugs fixed” cannot be objectively assessed and may cost you marks in both sections. Write criteria that you can later evaluate against your test results.

Plan before you test. Your Jira board should show that this section was produced first, not written up afterwards to match what happened.

4.2. Selenium Automation

Automate end-to-end tests of the application with Selenium WebDriver.

- Use a Maven project, Java 17 or later, IntelliJ IDEA, Selenium 4, and Chrome.
- All Selenium code must be inside `src/test/java`. Nothing testing-related belongs in the main source folder.
- Use one test class per scenario area, each with setup (`@BeforeAll` or `@BeforeEach`), steps (`@Test`), and teardown (`@AfterAll`).
- Cover at least 05 scenario areas. `Visit Services and Billing` must be one of them.
- Assert with JUnit 5. Use a range of assertions and a range of locator strategies, and check the outcome of each test, not only that the steps ran.
- Include at least one data-driven test using `@ParameterizedTest` that runs the same test over several sets of input data.
- Synchronise with explicit waits (`WebDriverWait`). `Thread.sleep` is not accepted as a synchronisation strategy.
- Prefer stable locators such as `id`, `name`, or a considered CSS selector. Absolute `XPath` copied from browser developer tools produces brittle tests and scores poorly.

At least one test must fail because it detects behaviour that contradicts the specification, not because a step broke or a wait timed out. If every test passes, treat that as a warning sign and revisit your assertions rather than as evidence that the release is clean.

Copy the Surefire reports produced by your final run into a `reports/` folder inside the project and submit them with the project. Exclude the `target` folder itself.

4.3. BDD Acceptance Testing with Katalon

Express the acceptance criteria the clinic cares about as executable specifications, using feature files and step definitions in Katalon Studio.

- Write 02 feature files. One of them must cover `Visit Services and Billing`.
- Write 04 to 06 scenarios in total across the two files. A Scenario Outline counts as one scenario.
- Include at least 01 Scenario Outline with an Examples table of 03 or more rows.
- Include at least 01 negative scenario covering invalid input or a rejected action.
- Include at least 01 scenario that states the behaviour the specification requires and that currently fails on the release candidate.
- Reuse at least one step definition across two or more scenarios.

Write your steps declaratively, describing business behaviour rather than user interface mechanics. *Given the owner has a pet with no recorded visits* is a business step. *Given I click the Add Visit button and then click Save* describes user interface actions rather than business behaviour and will score in the lowest band. A

clinic manager who does not use your testing tools should be able to read your feature files and confirm that they describe the intended behaviour. Keep specific values out of the step text and place them in the Examples table instead, so that the same step can be reused with different data.

Every scenario must trace to an acceptance criterion of a user story in your Jira project. State that mapping in your report. Export the Katalon execution report and submit it with the project.

4.4. Performance Testing with JMeter

Assess how the application behaves as the number of concurrent users increases. Build a single JMeter test plan (one JMX file) with the following configuration:

- Thread groups for 1, 10, 25, and 50 users, with a ramp-up period of 10 seconds, scheduled for 20 seconds.
- A constant timer with a delay of 300 milliseconds.
- Continue on error set for every thread group.
- Samplers covering at least 04 pages, including at least one page that triggers a billing calculation.
- A Response Assertion on each page, so that a fast response returning the wrong content is not recorded as a success.
- View Results Tree, Aggregate Report, Graph Results, and Summary Report listeners.
- Results saved to a `.jtl` file in CSV format and submitted with the test plan.

Describe your test environment in the report. State the machine, operating system, processor and memory, Java version, how the application was started, whether JMeter ran on the same machine, and what else was running at the time. Performance figures without an environment description cannot be interpreted, and the quality of this description will be assessed.

Write an analysis of at most 02 pages. Reproduce the Aggregate Report figures as a table in the report, typed out rather than provided only as a screenshot. Compare the four load levels in terms of throughput per second; minimum, maximum, and average response times; and error rate.

Interpret your results in relative terms. You are measuring a local development deployment backed by an in-memory database, so what your data can support is a comparison between load levels and a statement about the trend and where it starts to degrade. Conclusions about how many real users the clinic could serve are not supported by this setup, and making such claims will cost marks.

4.5. Test and Defect Management in Jira

Run the engagement through a single Jira project with the QAlity app. This is how your team coordinates, and it is also the record of how the work happened.

- Write at least 05 user stories with acceptance criteria for the areas you test.
- Create your test cases in QAlity, mapped to those acceptance criteria. Create at least one test cycle, execute it, and update the status of each test case.
- Raise every discrepancy you find as a defect in Jira, with a clear summary, steps to reproduce, the expected result citing the relevant requirement, the actual result with the exact value or behaviour observed, a severity and a priority each with a one-line justification, supporting evidence, and a link to the test that detected it.
- Triage your defects. Every defect must have an owner and a current status. A board of open tickets that nobody has assigned or moved is a list, not test management.

- Assign work to named members and keep the board current as the work happens.

Include one traceability table in your report running from user story to acceptance criterion to test case to the automation that covers it (a Selenium test, a Gherkin scenario, or a JMeter sampler) to any defect raised. This table is where the four workstreams are shown to be one engagement rather than four separate exercises.

Export your Jira issue list as a CSV file and submit it. Marks in this section reflect whether the board shows genuine coordination over the life of the project. Creating every issue in the final week does not show coordination, whatever the issue count.

4.6. Release Recommendation and Test Summary

Bring the engagement to a conclusion the client can act on.

- Summarise what was tested, by which member, and with what coverage, and state what was not tested and why.
- Summarise the defects you found by severity, and identify the ones that matter most to this clinic.
- Judge the release against the exit criteria you set in Section 4.1, stating for each criterion whether it was met.
- Give a clear recommendation: Release, Release with conditions, or Do not release. If there are conditions or blockers, name them.
- State the residual risk: What could still go wrong despite the testing you have performed.

Your recommendation must follow from your own evidence. A recommendation that contradicts your exit criteria, your defect list, or your performance data will score poorly however well it is written. This section is where your judgement is assessed, so a confident recommendation with reasoning behind it is worth more than a summary that avoids taking a position.

4.7. Presentation

Present your findings to the client in the Week 12 Tutorial.

- 15 minutes of presentation, followed by 05 minutes of client questions.
- Submit your slides as a PDF file of at most 15 slides, inside the submission zip.
- No live demonstration. Use screenshots or short embedded clips instead.

This is a report to a client, not a walkthrough of your submission. Cover your scope and key assumptions, your approach and why you tested what you tested first, what you found and what it means for the clinic, your performance findings and how you read them, and your recommendation with its residual risk.

Every member presents the area they owned. A member who is absent without approved special consideration receives no marks for this criterion. Marks are awarded for explaining why you did something and what it means, not simply for listing what you did.

4.8. AI Usage and Critique (About One Page)

- **Declaration:** A short table listing each AI tool used, its model or product name, what it was used for, and which member used it. If your group used no AI tools, state this explicitly.
- **Critique:** Choose one substantive AI contribution, for example a generated Selenium test class, a generated set of Gherkin scenarios, a suggested test plan section, or a JMeter configuration. Include the prompt and the output, with long output in an appendix. Then identify at least 02

places where the AI output was wrong, misleading, or unusable once you checked it against the Requirements Summary or the running application. Show the evidence and explain what you changed and why.

A critique that could have been written without running the application will receive few marks. The point is to show that your group verified the AI, not that it trusted it.

5. REPORT STRUCTURE

Each section maps to the rubric criterion of the same name.

- **Cover Page:** Your group number, the full name and student number of every member, a link to your Jira project, and a link to your group GitHub repository.
- 1. Test Planning and Assumptions.
- 2. Selenium Automation.
- 3. BDD Acceptance Testing with Katalon.
- 4. Performance Testing with JMeter.
- 5. Test and Defect Management in Jira.
- 6. Release Recommendation and Test Summary.
- 7. Presentation (A half page stating who presented which area, with your slide file named).
- 8. AI Usage and Critique.
- **Appendix A (Required):** The contribution table described in Section 06, using the template at the end of this brief.
- **Further Appendices (Optional):** Screenshots, full test case specifications, full AI transcripts, and full JMeter output.

The main body (Sections 1 to 8) must not exceed 20 pages, using 11-point Calibri, 1.25 line spacing, and standard margins (about 2 cm). Submit the report in black and white. Appendices do not count toward the page limit, but your analysis, your reasoning, and your recommendation must be in the main body, because markers assess the main body against the rubric and use the appendices only as evidence.

6. CONTRIBUTION

6.1. Version Control

Your group keeps all of its code in a single GitHub repository. This is how a test team works, and it is also the record that shows who did what.

- Create one private GitHub repository for your group and keep it private for the whole semester. A public repository exposes your work to other groups and is treated as a collusion risk.
- Add `honguyenphubao@gmail.com` as a collaborator when you create the repository.
- Commit the Selenium project, the Katalon project, and the JMeter test plan to that repository as you build them.
- Every member commits their own work under their own GitHub account. Do not have one member push everybody else's files.

The repository is not assessed as a separate criterion. It serves as evidence if the contribution table in Appendix A is disputed. A history showing work distributed across members and over time supports your

contribution claims; a single upload near the deadline does not. A missing or inaccessible repository is treated as a submission breach under Section 07.

6.2. Contribution Factor

Your individual mark is your group mark multiplied by a contribution factor.

- The factor defaults to 1.0 and is capped at 1.0. It can reduce an individual mark but never increase one, so a group in which everyone contributed fully has nothing to do beyond recording that.
- Your group agrees the factors and records them in Appendix A, using the template at the end of this brief. All 05 members confirm the table by name.
- If your group agrees, the table is accepted as submitted and no further evidence is required.
- If your group cannot agree, or if the evidence contradicts the table, the course coordinator determines the factors using the Jira history, the GitHub commit history, and the client questions in Section 4.7.

Contribution is judged by what your group could actually use, not by how much activity a member generated. A large number of Jira issues or commits is not evidence of contribution. Work that had to be redone by someone else, or that nobody used in the final submission, counts for little, and a small number of substantial pieces counts for more than many fragments.

If you have a concern about contribution that you do not wish to raise inside your group, contact the course coordinator before the due date.

7. SUBMISSION INSTRUCTIONS

Submit a single `.zip` file on Canvas named `[GroupNumber]-A3.zip`. Only one member of the group submits. The zip must contain the following, using exactly this structure.

- `[GroupNumber]-A3-Report.docx`.
- `[GroupNumber]-A3-Slides.pdf`.
- `Selenium/`, Maven Project, excluding `target` folder and including `reports/` folder.
- `Katalon/`, Katalon Project, including feature files and step definitions.
- `JMeter/`, containing `[GroupNumber].jmx` and your `.jtl` results file in CSV format.
- `Exports/`, containing your Jira CSV export and your Katalon execution report.

Do not submit `.rar` archives, do not nest a `zip` inside the `zip`, and do not replace any required file with a cloud storage link. Every piece of evidence must be inside the single `zip`. Submissions that do not follow these instructions, for example the wrong file format, an incorrect file or `zip` name, or files placed outside the single `zip`, will be penalised.

8. ACADEMIC INTEGRITY AND USE OF AI

You may use AI tools on this assignment, and using them well is a skill this course values. AI is here to support your work, not to do it for you. Your group must understand, and be able to justify, everything it submits: The plan you wrote, the tests you automated, the scenarios you specified, the figures you measured, and the recommendation you made. Work produced by an AI tool and handed in without your own understanding will not earn marks, and Sections 4.7 and 4.8 are where that understanding is checked.

All submitted work must be your group's own. Sharing projects, test cases, or reports with another group, or submitting evidence taken from another group's work, is not permitted. Your screenshots and results must come from your own Jira, QAlity, Katalon, IntelliJ, and JMeter sessions. Source code and reports are checked for similarity. Standard RMIT academic integrity rules apply.

9. QUESTIONS

Review the questions already answered in the Canvas forum before posting a new one. Post course-related questions to the forum so that all students can see the answer.

10. APPENDIX A: CONTRIBUTION TABLE TEMPLATE

Complete one row per member and place the finished table in Appendix A of your report. Areas Owned refers to the report sections in Section 05 that the member led. Key Testing Decision Made is one decision the member made themselves and can defend during the client questions, for example a choice of locator strategy, a load profile, a severity rating, or an assumption recorded in the test plan.

Member (Name & Student ID)	Areas Owned	Main Deliverables Produced	Key Testing Decision Made	Contribution Factor

Give a one-line justification below the table for any factor lower than 1.0. If your group cannot reach agreement, submit the table with the factors you do agree on, note the point of disagreement below it, and contact the course coordinator before the due date.